

2 Example Task 5680 PR Triage & Impact Assessment

Example Task 5680

Name: milvus-io-milvus-38876-pr
Intent: PR Triage & Impact Assessment
Difficulty: Medium
Language: Go

Repo

Repository: <https://github.com/milvus-io/milvus>

Checkout (commit): 76c66f07be6b04da93b5e722c8dc2fadd98daef6

Prompt

What is the change in pr.patch actually doing under the covers, beyond the headline "unblock previously-skipped Go client search tests"? Which parts of the milvus Go client test suite and the search-parameter API surface ripple from it? And what concrete issues should I flag to the author before I approve this for merge?

pr.patch (this exists on disk only; do not paste into prompt)

```
diff --git a/tests/go_client/testcases/groupby_search_test.go
b/tests/go_client/testcases/groupby_search_test.go
index 72b8a7b8bdc4b..7ad196d633242 100644
--- a/tests/go_client/testcases/groupby_search_test.go
+++ b/tests/go_client/testcases/groupby_search_test.go
@@ -90,7 +90,6 @@ func prepareDataForGroupBySearch(t *testing.T, loopInsert
int, insertNi int, idx
    // -> verify every top passage is the top of whole group
    // output_fields: pk + groupBy
    func TestSearchGroupByFloatDefault(t *testing.T) {
-        t.Skip("https://github.com/milvus-io/milvus/issues/38343")
        t.Parallel()
        for _, idx := range genGroupByVectorIndex(entity.L2) {
            // prepare data
@@ -148,7 +147,6 @@ func TestSearchGroupByFloatDefault(t *testing.T) {
    }

    func TestSearchGroupByFloatDefaultCosine(t *testing.T) {
-        t.Skip("https://github.com/milvus-io/milvus/issues/38343")
        t.Parallel()
        for _, idx := range genGroupByVectorIndex(entity.COSINE) {
            // prepare data
@@ -297,7 +295,6 @@ func TestSearchGroupByBinaryDefault(t *testing.T) {
```

```

// default Bounded ConsistencyLevel -> succ ??
// strong ConsistencyLevel -> error
func TestSearchGroupByBinaryGrowing(t *testing.T) {
-   t.Skip("https://github.com/milvus-io/milvus/issues/38343")
    t.Parallel()
    for _, metricType := range hp.SupportBinIvfFlatMetricType {
        idxBinIvfFlat := index.NewBinIvfFlatIndex(metricType, 128)
@@ -329,6 +326,7 @@ func TestSearchGroupByBinaryGrowing(t *testing.T) {

// groupBy in growing segments, maybe growing index or brute force
func TestSearchGroupByFloatGrowing(t *testing.T) {
+   t.Parallel()
    for _, metricType := range hp.SupportFloatMetricType {
        idxHnsw := index.NewHNSWIndex(metricType, 8, 96)
        mc, ctx, collName := prepareDataForGroupBySearch(t, 100, 200,
idxHnsw, true)
@@ -382,6 +380,7 @@ func TestSearchGroupByFloatGrowing(t *testing.T) {

// groupBy + pagination
func TestSearchGroupByPagination(t *testing.T) {
+   t.Parallel()
        // create index and load
        idx := index.NewHNSWIndex(entity.COSINE, 8, 96)
        mc, ctx, collName := prepareDataForGroupBySearch(t, 10, 1000, idx, false)
@@ -442,19 +441,16 @@ func TestSearchGroupByIterator(t *testing.T) {

// groupBy + range search -> not supported
func TestSearchGroupByRangeSearch(t *testing.T) {
-   t.Skipf("Waiting for support for specifying search parameters")
+   t.Skipf("https://github.com/milvus-io/milvus/issues/38846")
        idxHnsw := index.NewHNSWIndex(entity.COSINE, 8, 96)
        mc, ctx, collName := prepareDataForGroupBySearch(t, 1, 1000, idxHnsw,
true)

        // groupBy search with range
        queryVec := hp.GenSearchVectors(common.DefaultNq, common.DefaultDim,
entity.FieldTypeFloatVector)

-   // sp, _ := index.NewHNSWIndexSearchParam(50)
-   // sp.AddRadius(0)
-   // sp.AddRangeFilter(0.8)
-
        // range search
-   _, err := mc.Search(ctx, client.NewSearchOption(collName,
common.DefaultLimit,
queryVec).WithGroupByField(common.DefaultVarcharFieldName).WithANNSField(common
.DefaultFloatVecFieldName))

```

```

+     _, err := mc.Search(ctx, client.NewSearchOption(collName,
common.DefaultLimit,
queryVec).WithGroupByField(common.DefaultVarcharFieldName)).
+
WithANNSField(common.DefaultFloatVecFieldName).WithSearchParam("radius",
"0").WithSearchParam("range_filter", "0.8"))
    common.CheckErr(t, err, false, "Not allowed to do range-search when doing
search-group-by")
}

diff --git a/tests/go_client/testcases/query_test.go
b/tests/go_client/testcases/query_test.go
index ca863fb4d60e3..d28c5cad4082 100644
--- a/tests/go_client/testcases/query_test.go
+++ b/tests/go_client/testcases/query_test.go
@@ -165,7 +165,6 @@ func TestQueryWithoutExpr(t *testing.T) {
    // test query with part not existed field ["aa", "$meat"]: error or as dynamic
    field
    // test query with repeated field: ["*", "$meat"], ["floatVec", floatVec"]
    unique field
    func TestQueryOutputFields(t *testing.T) {
-        t.Skip("verify TODO")
        t.Parallel()
        ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
        mc := createDefaultMilvusClient(ctx, t)
@@ -193,16 +192,17 @@ func TestQueryOutputFields(t *testing.T) {
        common.CheckErr(t, err1, false, "not exist")
    }

-        // query with not existed field -> output field as dynamic or error
+        // query with not existed field -> output empty data field as
    dynamic or error
        fakeName := "aaa"
        res2, err2 := mc.Query(ctx,
client.NewQueryOption(schema.CollectionName).WithConsistencyLevel(entity.ClStro
ng).WithFilter(expr).WithOutputFields(fakeName))
        if enableDynamic {
            common.CheckErr(t, err2, true)
            for _, c := range res2.Fields {
-                log.Debug("data", zap.String("name", c.Name()),
zap.Any("type", c.Type()), zap.Any("data", c.FieldData()))
+                log.Info("data", zap.String("name", c.Name()),
zap.Any("type", c.Type()), zap.Any("data", c.FieldData()))
            }
            common.CheckOutputFields(t,
[]string{common.DefaultInt64FieldName, fakeName}, res2.Fields)
-                dynamicColumn := hp.MergeColumnsToDynamic(10,
hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName)

```

```

+           dynamicColumn := hp.MergeColumnsToDynamic(10,
[]column.Column{}, common.DefaultDynamicFieldName)
+           // dynamicColumn := hp.MergeColumnsToDynamic(10,
hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName)
           expColumns := []column.Column{
               hp.GenColumnData(10, entity.FieldTypeInt64,
*hp.TNewDataOption()),
               column.NewColumnDynamic(dynamicColumn, fakeName),
diff --git a/tests/go_client/testcases/search_test.go
b/tests/go_client/testcases/search_test.go
index b84a5dda6cbe2..cb3e7373fb4ac 100644
--- a/tests/go_client/testcases/search_test.go
+++ b/tests/go_client/testcases/search_test.go
@@ -3,6 +3,7 @@ package testcases
import (
    "fmt"
    "math/rand"
+   "strconv"
    "testing"
    "time"

@@ -453,7 +454,6 @@ func TestSearchEmptyInvalidVectors(t *testing.T) {

    // test search metric type isn't the same with index metric type
    func TestSearchNotMatchMetricType(t *testing.T) {
-       t.Skip("Waiting for support for specifying search parameters")
        ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
        mc := createDefaultMilvusClient(ctx, t)

@@ -465,7 +465,7 @@ func TestSearchNotMatchMetricType(t *testing.T) {
        prepare.Load(ctx, t, mc, hp.NewLoadParams(schema.CollectionName))

        vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
-       _, errSearchEmpty := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, vectors))
+       _, errSearchEmpty := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithSearchParam("metric_type", "L2"))
        common.CheckErr(t, errSearchEmpty, false, "metric type not match: invalid
parameter")
    }

@@ -507,12 +507,6 @@ func TestSearchInvalidOffset(t *testing.T) {

    // test search with invalid search params
    func TestSearchInvalidSearchParams(t *testing.T) {
-       t.Skip("Waiting for support for specifying search parameters")
-    }

```

```

-
-// search with index hns w search param ef < topK -> error
-func TestSearchEfHns w(t *testing.T) {
-    t.Skip("Waiting for support for specifying search parameters")
    ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
    mc := createDefaultMilvusClient(ctx, t)

@@ -524,18 +518,29 @@ func TestSearchEfHns w(t *testing.T) {
    prepare.Load(ctx, t, mc, hp.NewLoadParams(schema.CollectionName))

    vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
-    _, err := mc.Search(ctx, client.NewSearchOption(schema.CollectionName,
common.DefaultLimit, vectors))
-    common.CheckErr(t, err, false, "ef(7) should be larger than k(10)")
-}

-// test search params mismatch index type, hns w index and ivf sq8 search param
-> search with default hns w params, ef=topK
-func TestSearchSearchParamsMismatchIndex(t *testing.T) {
-    t.Skip("Waiting for support for specifying search parameters")
+    // search with invalid hns w ef
+    invalidEfs := []int{-1, 0, 32769}
+    for _, invalidEf := range invalidEfs {
+        _, errHns w := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithSearchParam("ef", strconv.Itoa(invalidEf)))
+        common.CheckErr(t, errHns w, true, "No error for invalid search
params")
+    }
+
+    // test search params mismatch index type, hns w index and ivf sq8 search
param -> search with default hns w params, ef=topK
+    invalidNprobes := []int{-1, 0, 65537}
+    for _, invalidNprobe := range invalidNprobes {
+        _, errHns w := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithSearchParam("nprobe", strconv.Itoa(invalidNprobe)))
+        common.CheckErr(t, errHns w, true, "No error for invalid search
params")
+    }
+
+    // search with index hns w search param ef < topK -> error
+    res, err := mc.Search(ctx, client.NewSearchOption(schema.CollectionName,
common.DefaultLimit, vectors).WithSearchParam("ef", "7"))
+    common.CheckErr(t, err, true, "ef(7) should be larger than k(10), but no
error")
+    common.CheckSearchResult(t, res, 1, common.DefaultLimit)
+}

```

```

// search with index scann search param ef < topK -> error
func TestSearchInvalidScannReorderK(t *testing.T) {
-   t.Skip("Waiting for support for specifying search parameters")
    ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
    mc := createDefaultMilvusClient(ctx, t)

@@ -548,36 +553,49 @@ func TestSearchInvalidScannReorderK(t *testing.T) {
    prepare.Load(ctx, t, mc, hp.NewLoadParams(schema.CollectionName))

    // search with invalid reorder_k < topK
+   vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
+
+   // search with invalid hnsw ef
+   _, errScann := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, vectors).
+       WithSearchParam("nprobe", "8").WithSearchParam("reorder_k",
strconv.Itoa(common.DefaultLimit-1)))
+   common.CheckErr(t, errScann, true, "No error for invalid search params")

    // valid scann index search reorder_k
+   res, err := mc.Search(ctx, client.NewSearchOption(schema.CollectionName,
common.DefaultLimit, vectors).
+       WithSearchParam("nprobe", "8").WithSearchParam("reorder_k", "20"))
+   common.CheckErr(t, err, true)
+   common.CheckSearchResult(t, res, 1, common.DefaultLimit)
}

// test search with scann index params: with_raw_data and metrics_type [L2,
IP, COSINE]
func TestSearchScannAllMetricsWithRawData(t *testing.T) {
-   t.Skip("Waiting for support scann index params withRawData")
    t.Parallel()
-   /*for _, withRawData := range []bool{true, false} {
+   for _, withRawData := range []bool{true, false} {
        for _, metricType := range []entity.MetricType{entity.L2,
entity.IP, entity.COSINE} {
            ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout)
            mc := createDefaultMilvusClient(ctx, t)

-           prepare, schema := hp.CollPrepare.CreateCollection(ctx, t,
mc, hp.NewCreateCollectionParams(hp.Int64VecJSON), hp.TNewFieldsOption(),
hp.TNewSchemaOption())
-           prepare.InsertData(ctx, t, mc, hp.NewInsertParams(schema,
500), hp.TNewDataOption())
+           prepare, schema := hp.CollPrepare.CreateCollection(ctx, t,
mc, hp.NewCreateCollectionParams(hp.Int64VecJSON),

```

```

+             hp.TNewFieldsOption(),
hp.TNewSchemaOption().TWithEnableDynamicField(true))
+             prepare.InsertData(ctx, t, mc, hp.NewInsertParams(schema),
hp.TNewDataOption())
+             prepare.FlushData(ctx, t, mc, schema.CollectionName)
+             prepare.CreateIndex(ctx, t, mc,
hp.TNewIndexParams(schema).TWithFieldIndex(map[string]index.Index{
-             common.DefaultFloatVecFieldName:
index.NewSCANNIndex(entity.COSINE, 16),
+             common.DefaultFloatVecFieldName:
index.NewSCANNIndex(metricType, 16, withRawData),
+             }))
+             prepare.Load(ctx, t, mc,
hp.NewLoadParams(schema.CollectionName))

+             // search and output all fields
-             vectors := hp.GenSearchVectors(1, common.DefaultDim,
entity.FieldTypeFloatVector)
-             resSearch, errSearch := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
vectors).WithConsistencyLevel(entity.ClStrong).WithOutputFields([]string{"*"}))
+             vectors := hp.GenSearchVectors(common.DefaultNq,
common.DefaultDim, entity.FieldTypeFloatVector)
+             resSearch, errSearch := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, vectors).
+             WithConsistencyLevel(entity.ClStrong).WithOutputFields("*"))
+             common.CheckErr(t, errSearch, true)
-             common.CheckOutputFields(t,
[]string{common.DefaultInt64FieldName, common.DefaultFloatFieldName,
-             common.DefaultJSONFieldName,
common.DefaultFloatVecFieldName, common.DefaultDynamicFieldName},
resSearch[0].Fields)
-             common.CheckSearchResult(t, resSearch, 1,
common.DefaultLimit)
+             common.CheckOutputFields(t, []string{
+             common.DefaultInt64FieldName,
common.DefaultJSONFieldName,
+             common.DefaultFloatVecFieldName,
common.DefaultDynamicFieldName,
+             }, resSearch[0].Fields)
+             common.CheckSearchResult(t, resSearch, common.DefaultNq,
common.DefaultLimit)
+             }
-             }*/
+             }
+             }

+             // test search with valid expression

```



```

@@ -1061,7 +1079,7 @@ func TestSearchSparseVectorNotSupported(t *testing.T) {
}

func TestRangeSearchSparseVector(t *testing.T) {
-   t.Skip("Waiting for support range search")
+   t.Skipf("https://github.com/milvus-io/milvus/issues/38846")
   ctx := hp.CreateContext(t, time.Second*common.DefaultTimeout*2)
   mc := createDefaultMilvusClient(ctx, t)

@@ -1072,5 +1090,30 @@ func TestRangeSearchSparseVector(t *testing.T) {
   prepare.InsertData(ctx, t, mc, hp.NewInsertParams(schema),
hp.TNewDataOption().TWithSparseMaxLen(128))
   prepare.FlushData(ctx, t, mc, schema.CollectionName)

-   // TODO range search
+   // range search
+   queryVec := hp.GenSearchVectors(common.DefaultNq, common.DefaultDim,
entity.FieldTypeSparseVector)
+
+   resRange, errSearch := mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit,
queryVec).WithSearchParam("drop_ratio_search", "0.2"))
+   common.CheckErr(t, errSearch, true)
+   require.Len(t, resRange, common.DefaultNq)
+   for _, res := range resRange {
+       log.Info("default search", zap.Any("score", res.Scores))
+   }
+
+   radius := 10
+   rangeFilter := 30
+   resRange, errSearch = mc.Search(ctx,
client.NewSearchOption(schema.CollectionName, common.DefaultLimit, queryVec).
+       WithSearchParam("drop_ratio_search",
"0.2").WithSearchParam("radius",
strconv.Itoa(radius)).WithSearchParam("range_filter",
strconv.Itoa(rangeFilter)))
+   common.CheckErr(t, errSearch, true)
+   common.CheckErr(t, errSearch, true)
+   require.Len(t, resRange, common.DefaultNq)
+   for _, res := range resRange {
+       log.Info("range search", zap.Any("score", res.Scores))
+   }
+   for _, res := range resRange {
+       for _, s := range res.Scores {
+           require.GreaterOrEqual(t, s, float32(10))
+           require.Less(t, s, float32(30))
+       }
+   }
}

```

LLM Prompt Review

Prompt QC Feedback

****Grading Item 1 (1/1)****

The question requests an in-depth explanation of a patch's effect, indicating a non-trivial analysis.

****Grading Item 2 (1/1)****

It reads as a realistic peer-to-peer developer question without role-play language.

****Grading Item 3 (1/1)****

It avoids asking for elementary definitions or generic explanations.

****Grading Item 4 (1/1)****

Understanding the change, its ripple effects, and issues requires substantial repo exploration.

****Grading Item 5 (1/1)****

The prompt is purely interrogative and contains no inline repo content.

****Grading Item 6 (1/1)****

The question is specific and clearly grounded in repo context with verifiable answers.

****Grading Item 7 (1/1)****

It aligns with the task intent of analyzing repository changes.

****Grading Item 8 (1/1)****

It demands reasoning about system behavior, impacts, and potential issues.

****Grading Item 9 (1/1)****

No meta-instructions or format directives appear.

****Grading Item 10 (1/1)****

All characters are ASCII.

FINAL_SCORE: 10

Rubric

ID	tag	description	weight	rationale	dependent_on
69396	reasoning	Identifies that three previously-skipped tests in	critical	The un-skipping is the headline activation in the GroupBy file.	

		tests/go_client/testcases/groupby_search_test.go lose their issue-38343 t.Skip guards (TestSearchGroupByFloatDefault, TestSearchGroupByFloatDefaultCosine, TestSearchGroupByBinaryGrowing).		Anchoring the triage on the three named tests is how the reader connects the issue-38343 reference to the concrete CI coverage that just turned on.	
69397	reasoning	Identifies that TestSearchSearchParamsMismatchIndex is deleted from search_test.go.	critical	TestSearchSearchParamsMismatchIndex was a t.Skip-only stub at the base commit (body == one t.Skip line). Its deletion is half of the consolidation that reduces three function names to one. Naming this deletion is what bounds the consolidation claim on the smaller of the two donors.	
69398	reasoning	Identifies that TestSearchEfHnsw is deleted from search_test.go.	critical	TestSearchEfHnsw at the base commit had a full body of setup plus a search asserting CheckErr(t, err, false, 'ef(7) should be larger than k(10)'), all gated behind a leading t.Skip. Naming the deletion (separately from the merge into TestSearchInvalidSearchParams) is what lets the reviewer track which donor function carried the original error-direction assertion that gets inverted post-merge.	
69399	reasoning	Identifies that the new TestSearchInvalidSearchParams body absorbs the setup scaffolding (ctx, mc, prepare, schema, prepare.Load) plus search loops from the deleted TestSearchEfHnsw.	critical	The merge is what makes the consolidation coherent. Without naming the merge a reader sees only function-name deletions, missing the through-line that the new TestSearchInvalidSearchParams body absorbs the donor's work. The previous TestSearchInvalidSearchParams body contained only a t.Skip line.	69398
69400	reasoning	Identifies that the new TestSearchInvalidSearchParams asserts success for invalid ef values in {-1, 0,	critical	The CheckErr helper at response_checker.go:19-41 treats the second argument as	

		32769} via CheckErr called with expErrNil=true.		expErrNil. When expErrNil is true the helper calls require.NoError. The third argument is then an unused message. The new body issues searches with invalidEfs {-1, 0, 32769} where each call asserts no error. The test name promises invalid-param coverage. The assertion direction documents silent acceptance. Without surfacing this contradiction a reviewer ships an assertion direction that points the wrong way.	
69401	reasoning	Identifies that the new TestSearchInvalidSearchParams asserts success for invalid nprobe values in {-1, 0, 65537} via CheckErr called with expErrNil=true.	critical	The nprobe loop is the second of two assertion-direction inversions in the same body. Catching only the ef loop misses half of the silent-acceptance pattern. The nprobe parameter exercises a different index family than ef, so it is not redundant evidence of the same call path.	
69402	reasoning	Identifies that the ef=7 case in TestSearchInvalidSearchParams asserts success even though DefaultLimit=10 (the historical validation rule required ef > k).	critical	The pre-PR equivalent assertion was CheckErr(t, err, false, 'ef(7) should be larger than k(10)') which expected the error. The post-PR assertion flips to expErrNil=true with message 'ef(7) should be larger than k(10), but no error', so the message documents the behaviour change in plain prose. A reviewer who reads only the literal CheckErr call without unpacking the helper semantics misses the inverted assertion entirely.	69400
69403	reasoning	States that the PR makes no changes to production code under client/, internal/, or pkg/.	critical	Anchoring the triage in the test-only scope bounds the severity ceiling. A reviewer who imagines production-side risk inflates the severity, asking for review of code that does not	

				move in this patch.	
69404	reasoning	Identifies the migration to the new WithSearchParam(key, value) builder API on searchOption defined in client/milvusclient/read_options.go.	critical	The reactivated and new tests reach the production search-param wire format through .WithSearchParam(...) calls (radius, range_filter, metric_type, ef, nprobe, drop_ratio_search, reorder_k). Naming the builder is what connects the un-skip pattern to the actual API unlock the PR exercises for the first time in this acceptance path.	
69405	reasoning	Identifies that TestSearchGroupByRangeSearch is re-skipped under issue 38846 (its t.Skip message rotates from 'Waiting for support for specifying search parameters' to a #38846 reference).	critical	The re-skip pattern is easy to miss because the t.Skip line is preserved. A reviewer who reads only the skip line treats this test as inactive without noticing that the issue reference has rotated from a generic message to a specific tracking issue.	
69406	reasoning	Identifies that TestRangeSearchSparseVector is re-skipped under issue 38846 (its t.Skip message rotates from 'Waiting for support range search' to a #38846 reference).	critical	The same re-skip pattern lands a second time in search_test.go. Catching only the GroupBy variant misses the parallel rotation in the sparse-vector test. Both re-skips share the #38846 follow-up but live in different test files, so they are independently observable.	
69407	reasoning	Identifies that the body of TestSearchGroupByRangeSearch is migrated to the new WithSearchParam API.	critical	The body-migration-under-skip pattern is what would surprise a future contributor who closes #38846 expecting an empty body, instead landing a fully-implemented assertion on first un-skip. Without naming the migrated body a reviewer treats the re-skip as a no-op rather than as staged work. The migrated call replaces a commented-out sp.AddRadius/sp.AddRangeFilter with	

				.WithSearchParam('radius', '0').WithSearchParam('range_filter', '0.8').	
69408	reasoning	Identifies that the body of TestRangeSearchSparseVector contains a WithSearchParam-based range search asserting per-score bounds in [10, 30).	critical	The sparse-vector test ships the most substantive body-under-skip in this PR (per-score assertions inside a half-open interval). A future contributor who closes #38846 inherits live assertions on first un-skip. Catching this body separately from the GroupBy variant matters because the assertions check actual returned scores rather than only call success.	69406
69409	reasoning	Identifies the semantic flip in TestQueryOutputFields where MergeColumnsToDynamic switches from GenDynamicColumnData(0, 10) to an empty []column.Column{} slice.	critical	MergeColumnsToDynamic at data_helper.go:433 iterates the columns slice per row to assemble each row's dynamic JSON object. The empty slice yields empty per-row maps. The test moves from asserting a populated dynamic-field response (ten rows of generated data) to asserting empty dynamic data when the requested output field name does not exist. This is a spec change rather than a refactor, so it warrants a one-line note from the author.	
69410	reasoning	Notes the new three-argument NewSCANNIndex(metricType, 16, withRawData) wiring in TestSearchScannAllMetricsWithRawData.	bonus	The pre-PR call was index.NewSCANNIndex(entity.COSINE, 16) inside the skipped /* ... */ block. The current production signature at client/index/scann.go:44 takes three arguments. Naming the wire-up shows the reviewer understood why the test could only run after the skip lifted: the body's prior 2-arg call would not compile against current production.	
69411	reasoning	Notes the dead commented-out	bonus	Leaving a one-line comment	69409

		MergeColumnsToDynamic call left in TestQueryOutputFields immediately above the new live call.		that mirrors the previous behaviour adds no value, rotting quickly. A reviewer who flags the dead-code line is doing the lowest-cost housekeeping ask the PR would benefit from. The observation also pairs well with the wider concern that the semantic flip in this test needs a PR-description note.	
69412	reasoning	Notes the duplicate common.CheckErr(t, errSearch, true) line on consecutive lines inside TestRangeSearchSparseVector.	bonus	The duplicate is a clear copy-paste artefact. It runs twice harmlessly today but degrades readability, surfacing if someone changed one of the two without realising they had a sibling. A reviewer who catches the duplicate is reading the body of a re-skipped test rather than skipping past it on the t.Skip line.	
69413	reasoning	Notes that the 'but no error' suffix in the ef=7 CheckErr message wording documents an observed behaviour change.	bonus	The message text only renders when CheckErr is invoked with expErrNil=false where an error is present. Because expErrNil is true here, the message is a dead string. The author's wording ('ef(7) should be larger than k(10), but no error') is therefore documentation of a behaviour change the author observed, not an assertion of it. The smoking-gun observation distinguishes a careful read of the helper semantics from a surface read of the CheckErr call.	69402
69414	reasoning	Notes the log.Debug to log.Info change inside TestQueryOutputFields.	bonus	Promoting the per-row dynamic-field dump from debug to info level means CI runs will surface the row dump in standard log output going forward. Whether that is wanted is a one-line question from the author. A reviewer	

				who flags it is reading the diff for noise-level changes rather than only assertion changes.	
69415	reasoning	Names a specific path under client/, internal/, or pkg/ as modified by this PR.	penalty	The diff is restricted to tests/go_client/testcases/group by_search_test.go, query_test.go, search_test.go. Naming any non-test path as modified misreads the diff, routing the review toward production layers that do not move in this patch. The penalty fires on a fabricated specific surface rather than on a missing general scope claim, so it captures a positive misstatement instead of an omission.	
69416	reasoning	Cites a specific authentication, authorisation, RBAC, or untrusted-input handler as affected by the diff under review.	penalty	No such file is touched by the PR. Citing one fabricates a security implication the diff does not carry, inflating the severity of a test-only change. The penalty targets a positive false claim about a specific code path rather than an omitted security framing.	
69417	reasoning	Recommends reverting the un-skip changes as the appropriate fix for the inverted-assertion concern in TestSearchInvalidSearchParams.	penalty	The un-skips are correct work. Only the assertion direction inside the new body is the question. The right fix is to flip the second CheckErr argument from true to false if the intent is to assert rejection, or to rename the test if the intent is to document silent acceptance. Reverting throws away the consolidation, plus the migration to the new builder API.	
69418	reasoning	Recommends preserving the deleted stubs (TestSearchEfHnsw, TestSearchSearchParamsMismatchIndex) for git-blame continuity.	penalty	Skip-only stubs carry no executable coverage and keeping them blocks the consolidation that makes the merged TestSearchInvalidSearchPara	

				ms body the right review target.	
69419	reasoning	Recommends redesigning the CheckErr helper or the WithSearchParam API as the primary triage outcome.	penalty	A redesign-first answer skips the actual review job. The change is a concrete diff to approve or request-changes on, not a test-helper or builder-API exercise. Recommending a redesign leaves the inverted-assertion question unanswered.	
69420	reasoning	Recommends splitting the un-skip work from the consolidation refactor as a hard blocker.	penalty	Splitting at review time adds two PRs of overhead without producing a behaviour change. The consolidation is what makes the un-skips coherent. The right outcome is to ask the author to call out the consolidation in the PR description, not to demand a split.	
69421	completeness	Enumerates the three test files changed by the diff (tests/go_client/testcases/groupby_search_test.go, query_test.go, search_test.go).	critical	The mechanical surface of the PR spans three files under tests/go_client/testcases. A triage that does not enumerate the files is incomplete because downstream blast-radius/severity claims rest on knowing which surfaces moved.	
69422	completeness	Names the WithSearchParam builder API in client/milvusclient/read_options.go as a production surface reached by the reactivated tests but not modified by this PR.	critical	The reactivated tests reach the production search-param wire format through .WithSearchParam(...) calls. Naming the builder is what grounds the no-production-change claim against the actual API the PR exercises for the first time in this acceptance path.	
69423	completeness	Names NewSCANNIndex in client/index/scann.go as a production surface reached by the reactivated TestSearchScannAllMetricsWithRawData but not modified by this PR.	critical	The pre-PR test body called the two-argument form which would not compile against current production. Naming the constructor is what shows the reviewer understood why the test could only run after the	

				skip lifted, while the production signature itself is left untouched.	
69424	completeness	Concludes the PR introduces no classic security regression in authentication, authorisation, RBAC, untrusted-input parsing, or cryptographic boundaries.	critical	Test-only PRs invite the lazy reading that drops the security analysis entirely. Explicitly concluding no classic regression is what makes the security framing complete, distinguishing a thorough triage from one that omits the security frame.	
69425	completeness	Includes an alternative-fixes paragraph that explicitly preempts at least one wrong-fix temptation from the set {reverting the un-skip changes, preserving the deleted stubs, splitting the un-skip from the consolidation}.	critical	Preempting wrong fixes is what makes the recommendation actionable. Showing the reviewer thought through alternatives before settling on a recommendation distinguishes a triage from a bullet list of complaints.	
69428	style	Uses plain-text identifiers for function names in the response prose.	bonus	Plain-text function names help the reader visually parse references at a glance, avoiding the generated-artifact feel of backtick-wrapped or monospace-wrapped tokens that breaks reading flow inside a prose review.	
69429	style	Uses plain-text identifiers for type names in the response prose.	bonus	Plain-text type identifiers keep symbol formatting uniform across the response. Mixing backtick-wrapped types with plain-text functions creates visual inconsistency that distracts the reader from the analysis itself.	
69430	style	Uses double-quoted file paths in the response prose.	bonus	Double-quoted file paths visually separate path references from surrounding prose. Unquoted paths blur into ordinary words (especially short relative paths like cmd/ or pkg/), making the reader work harder to spot file references inside the analysis.	

LLM Rubric Review

Rubric QC Feedback

****Grading Item 1: The rubric is organized into exactly three top-level evaluation categories... (2/2)****

All three required categories—reasoning, completeness, and style—are present with no extras.

****Grading Item 2: The rubric contains a reasonable number of grading items... (2/2)****

Thirty-three grading items appropriately match the task complexity and cover the space without feeling sparse or bloated.

****Grading Item 3: Every grading item is atomic and self-contained... (2/2)****

Each item targets a single clearly defined expectation; none stack multiple unrelated checks together.

****Grading Item 4: Grading item descriptions are clear, specific, and measurable... (2/2)****

Descriptions cite precise tests, functions, or behaviors, making compliance straightforward to assess.

****Grading Item 5: Every grading item is derivable from the provided `golden\_answer` ... (2/2)****

All requirements trace back to facts or emphases explicitly present in the golden answer.

****Grading Item 6: The rubric is comprehensive relative to the `golden\_answer` ... (2/2)****

Major themes—test migrations, assertion inversions, API surfaces, and follow-up concerns—are all represented with appropriate weight.

****Grading Item 7: The rubric would award full marks to the provided `golden\_answer` ... (2/2)****

The golden answer satisfies every critical requirement and avoids all penalties, so it would score full points.

****Grading Item 8: The rubric is fair to valid alternative reasoning paths... (2/2)****

While detailed, the items allow any accurate repo-grounded explanation to earn credit; they do not demand a single phrasing.

****Grading Item 9: The `reasoning` category evaluates correctness of reasoning, technical judgment... (2/2)****

Reasoning items focus on interpreting code changes, their implications, and technical judgments rather than mere coverage.

****Grading Item 10: The `completeness` category evaluates whether the response addresses all major parts... (2/2)****

Completeness items ensure scope, touched files, API surfaces, and alternative-fix coverage, matching the task demands.

****Grading Item 11: The `style` category evaluates clarity, organization, readability... (2/2)****

Style items address presentation aspects—identifier formatting and path quoting—without mixing in technical correctness.

****Grading Item 12: Grading items are placed in appropriate categories... (2/2)****

Each requirement resides in the logical category (reasoning for analysis points, completeness for coverage, style for prose).

****Grading Item 13: The rubric includes the task-wide expectations... (2/2)****

Reasoning quality, completeness, and style expectations are all explicitly represented.

****Grading Item 14: When the task intent requires it, the rubric includes appropriate intent-specific grading items... (2/2)****

Intent-specific needs—like naming wrong-fix temptations—are captured, aligning with the triage task.

****Grading Item 15: Weights use only valid values... and are well calibrated... (2/2)****

All items use critical/bonus/penalty appropriately, reserving penalties for harmful behavior and criticals for must-haves.

****Grading Item 16: Penalty items target genuinely harmful or misleading behavior... (2/2)****

Each penalty addresses a specific incorrect or misleading recommendation rather than being a simple negation of a positive requirement.

****Grading Item 17: Dependency usage is logically sound... (2/2)****

Dependencies (e.g., needing the base observation before a derivative one) are used sparingly and correctly, with no inverted logic.

****Grading Item 18: The rubric is not overprescriptive... (2/2)****

Despite its detail, the rubric allows varied yet accurate explanations; it does not require parroting exact wording from the golden answer.

****Grading Item 19: Each grading item contributes meaningfully... (2/2)****

No redundant or low-value items are present; each adds a distinct evaluative axis tied to the task needs.

FINAL_SCORE: {38}

Golden Answer

The headline understates what the PR is doing. Underneath the "unblock previously-skipped tests" framing, this is a coordinated migration of the Go client acceptance suite onto a new WithSearchParam builder API, a consolidation that deletes one skip-only stub plus one fully-bodied but skipped helper test (folding the helper's body into a sibling), a re-skip of two scenarios under a different unresolved issue, and a set of new assertions that document a server-side behaviour change rather than verifying it. Production code does not move. The headline is broadly correct, but one block of new assertions reads as the opposite of what the test names promise, and that

is the most important thing to flag before merge. Notation throughout this triage: function names appear in plain text, type names appear in plain text, file paths are wrapped in double quotes, the analysis runs in prose with no fenced code blocks.

Code Change Summary

"tests/go_client/testcases/groupby_search_test.go" gets four kinds of edit. Three previously-skipped tests lose their `t.Skip("https://github.com/milvus-io/milvus/issues/38343")` guards and become active: `TestSearchGroupByFloatDefault`, `TestSearchGroupByFloatDefaultCosine`, and `TestSearchGroupByBinaryGrowing`. Two tests gain a `t.Parallel()` call: `TestSearchGroupByFloatGrowing` and `TestSearchGroupByPagination`. Every other GroupBy test in the file (`TestSearchGroupByFloatDefault` at line 94, `TestSearchGroupByFloatDefaultCosine` at line 152, `TestSearchGroupByBinaryDefault` at line 265, `TestSearchGroupByBinaryGrowing` at line 301) already calls `t.Parallel()` at the base commit and is not retouched, so after this PR every active GroupBy test in the file runs in parallel. `TestSearchGroupByRangeSearch` is re-skipped under a different issue (now `Skipf("https://github.com/milvus-io/milvus/issues/38846")`) but its body is rewritten in the process: the commented-out `sp.AddRadius(0)` and `sp.AddRangeFilter(0.8)` lines are deleted, and the search call is updated from the old `sp-style` search-param mechanism to the new chained `.WithSearchParam("radius", "0").WithSearchParam("range_filter", "0.8")` API. The body will not run until #38846 is resolved, but the API migration work is landed.

"tests/go_client/testcases/query_test.go" gets a single test reactivated and a small semantic flip inside it. `TestQueryOutputFields` loses its `t.Skip("verify TODO")` guard. Inside the `enableDynamic` branch, `log.Debug` is changed to `log.Info` so the debug-level row dump is now always printed, and the `dynamicColumn` construction is changed from `hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName)` to `hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)`. The old call is left in the file as a commented-out line. `MergeColumnsToDynamic` in

"tests/go_client/testcases/helper/data_helper.go" iterates the columns slice per row to assemble each row's dynamic JSON, and with an empty slice every row's map is empty. The test thereby moves from asserting that the query returns ten rows of populated dynamic data to asserting that querying a non-existent output field name returns the field with empty dynamic data per row.

"tests/go_client/testcases/search_test.go" gets the bulk of the change. A new import for `strconv` lands at the top. `TestSearchNotMatchMetricType` loses its skip and the search

call gains `.WithSearchParam("metric_type", "L2")` so the test now passes the metric type through the new builder rather than through the old default-config path. Two functions are deleted entirely: `TestSearchSearchParamsMismatchIndex` (which was a true skip-only stub at the base commit, body == one `t.Skip` line) and `TestSearchEfHnsw` (which already had a full body of setup plus a search asserting `common.CheckErr(t, err, false, "ef(7) should be larger than k(10)")`, all gated behind a leading `t.Skip`). The `TestSearchEfHnsw` body is what gets folded into `TestSearchInvalidSearchParams`, which previously contained only a `t.Skip` line. The new body of `TestSearchInvalidSearchParams` loops over `invalidEfs := []int{-1, 0, 32769}`, calling `mc.Search` with `WithSearchParam("ef", strconv.Itoa(...))` for each, then over `invalidNprobes := []int{-1, 0, 65537}` similarly, then issues one more search with `ef=7` (which is below `DefaultLimit=10`), and runs `CheckSearchResult` expecting one `nq` row of `DefaultLimit` results. `TestSearchInvalidScannReorderK` loses its skip and gains a body that issues an invalid search with `reorder_k=DefaultLimit-1` followed by a valid search with `reorder_k=20`. `TestSearchScannAllMetricsWithRawData` loses its skip and its surrounding `/* ... */` comment block, and the body is updated: the schema option gains `TWithEnableDynamicField(true)`, the insert no longer hardcodes a row count of 500, and the index constructor is updated from `index.NewSCANNIndex(entity.COSINE, 16)` to the new three-argument `index.NewSCANNIndex(metricType, 16, withRawData)` so both `withRawData=true` and `withRawData=false` get exercised across the L2/IP/COSINE metric types. The search query count rises from 1 to `common.DefaultNq=5` and the asserted output-field list drops `common.DefaultFloatFieldName`. `TestRangeSearchSparseVector` loses its old skip and is re-skipped under issue #38846 with a fully-implemented body: a default search using `WithSearchParam("drop_ratio_search", "0.2")`, then a range search adding `WithSearchParam("radius", "10").WithSearchParam("range_filter", "30")`, and finally per-score assertions that every returned score is in the half-open interval `[10, 30)`. The body contains a duplicate `common.CheckErr(t, errSearch, true)` line on consecutive lines, which is a copy-paste artefact.

That is the entire diff. No production code is touched.

Blast Radius

The change that ripples furthest is the wiring of the new `WithSearchParam` API. The builder method is defined twice in `"client/milvusclient/read_options.go"`: on the underlying `annRequest` at line 248 and on the public `searchOption` at line 354, both with signature `WithSearchParam(key, value string)`. Client-side validation does not exist, and the builder accumulates pairs into a `searchParam map[string]string` field at line 71 and merges them into the protobuf `SearchRequest's SearchParams` field via

entity.MapKvPairs(params) on the wire. Any rejection of bad values happens server-side. Every newly-active or migrated test in this PR is now reaching the server-side validator (or whatever should be acting as one) for the first time through this acceptance path.

The new SCANN tests reach "client/index/scann.go" at line 44 where NewSCANNIndex(metricType MetricType, nlist int, withRawData bool) is defined. Pre-PR the test code called the two-argument form which would not compile against current production, so the test was a compile-broken stub kept alive only by the t.Skip guard. The reactivation lines up the test with the production three-argument signature.

The reactivated TestQueryOutputFields reaches "tests/go_client/testcases/helper/data_helper.go" MergeColumnsToDynamic at line 433, and the new dynamic-column-empty assertion exercises a previously-unasserted shape: a Column whose per-row data is an empty JSON object. The test fakeName is a literal "aaa" and the test now pins what the query response looks like when the requested output field name does not exist on the schema.

The CheckErr helper in "tests/go_client/common/response_checker.go" lines 19 through 41 deserves special note because it shapes the most important triage observation in this PR. Its signature is CheckErr(t *testing.T, actualErr error, expErrNil bool, expErrorMsg ...string). When expErrNil is true the function calls require.NoError(t, actualErr), and the variadic expErrorMsg is then never inspected. When expErrNil is false the function calls require.Error and then checks that the error string contains one of the supplied substrings.

What does not ripple. Production search-param validation, the SCANN index implementation, the range-search support, the sparse-vector handler, and the dynamic-field codec all retain their pre-PR behaviour because the PR does not edit them. Test files outside the three named ones are not touched. The CI cost rises by the number of newly-active test bodies and is bounded by the t.Parallel additions on two of the GroupBy tests.

Severity and Complexity

I read this as Low-Medium. Test-only is a low ceiling, but one block of new assertions inverts the meaning the test names suggest, and that should not merge without an author confirmation.

Most consequential: the new bodies of `TestSearchInvalidSearchParams` and `TestSearchInvalidScannReorderK` assert success, not failure, for cases the test names describe as "invalid". Each `invalidEf` in `{-1, 0, 32769}` is exercised with `common.CheckErr(t, errHnsw, true, "No error for invalid search params")`. The `expErrNil=true` argument means `require.NoError`, and the "No error for invalid search params" string is the unused variadic arg. The same pattern repeats for `invalidNprobes` in `{-1, 0, 65537}`. After both loops the test runs one more search with `ef=7` (where `DefaultLimit=10` so `ef<topK` by the historical validation rule) and asserts `common.CheckErr(t, err, true, "ef(7) should be larger than k(10), but no error")`. The "but no error" suffix in that message wording is the smoking gun: the author knows `ef=7` used to error and is documenting that it no longer does.

`TestSearchInvalidScannReorderK` has the same shape, asserting success for `reorder_k = DefaultLimit - 1 = 9` against a `topK` of 10. Pre-PR the equivalent direct test asserted the opposite: `common.CheckErr(t, err, false, "ef(7) should be larger than k(10)")`. Three readings are possible. Either server-side validation of these parameters was removed elsewhere and this PR is documenting the change (which would be a production regression flagged in the wrong place), or the `WithSearchParam` plumbing does not reach the validator the way the older config-driven plumbing did (which would be a client-side wiring gap), or the test intent is to land a documentation-only assertion of current behaviour and leave the rejection assertion for follow-up. Any of the three needs to be named in the PR description. The fix if the intent was to assert rejection is to flip the second `CheckErr` argument from `true` to `false`, and if the intent is to assert current silent-acceptance, the test name and the third-arg messages should be reworded so a future reader does not read the asserts upside down.

Next: the `MergeColumnsToDynamic` change in `TestQueryOutputFields` is a semantic flip rather than a refactor. Pre-PR the test asserted that a query whose output-field list includes a non-existent name "aaa" returns the dynamic column populated with ten rows of generated dynamic data. Post-PR it asserts the dynamic column is returned with empty per-row maps. The old behaviour was either wrong or the spec changed, and the PR description does not say which. The right shape is to either restore the populated-data assertion (if the old behaviour was the intended spec) or document why the empty-data assertion is now correct. The leftover commented-out `hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), ...)` line should be removed before merge regardless because dead code in test files rots quickly.

Next: the re-skip of `TestSearchGroupByRangeSearch` and `TestRangeSearchSparseVector` under issue #38846 is reasonable, but the bodies are now fully implemented behind the skip. If a future contributor closes #38846 without re-reading the bodies, the range-search assertions (the score must be in `[10, 30)`, the

duplicate CheckErr line on consecutive lines) ship as part of the unblock. Cleaning up the duplicate CheckErr now costs one line and saves a future contributor a confused minute.

Smaller items. The t.Parallel additions to TestSearchGroupByFloatGrowing and TestSearchGroupByPagination bring the GroupBy tests in this file into uniform parallelism (every active GroupBy test in the file now calls t.Parallel) which is good housekeeping rather than a flag. log.Debug to log.Info in TestQueryOutputFields is a noise change that will surface row dumps in every CI run, and whether that is wanted is worth one line from the author. The new strconv import is paired only with WithSearchParam string-conversion calls, and if a future test wants to pass numeric params, the helper should be extracted instead of copying strconv.Itoa everywhere.

Alternative fixes that are tempting but wrong. Reverting the un-skip changes because the assertions look inverted is wrong: the un-skips are correct work, only the assertion direction inside TestSearchInvalidSearchParams is the question, and reverting throws away the consolidation of two stub functions plus the migration to the new builder API. Demanding the deleted TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex stubs be preserved for git-blame continuity is wrong, and deleting unused skip-only functions is housekeeping and the merge into TestSearchInvalidSearchParams is exactly the kind of consolidation a reviewer should encourage. Splitting the un-skip work from the consolidation refactor as a hard block is wrong because the consolidation is what makes the un-skips coherent, and splitting at review time adds two PRs of overhead without producing a behaviour change.

Recommended verdict: approve with comments. Required actions: confirm with the author whether TestSearchInvalidSearchParams and TestSearchInvalidScannReorderK intend to assert rejection or to document silent acceptance, and adjust the second CheckErr argument or the test names accordingly, remove the duplicate common.CheckErr(t, errSearch, true) line in TestRangeSearchSparseVector, and remove the commented-out MergeColumnsToDynamic line in TestQueryOutputFields.

Security Risk

No classic security regression. The diff does not introduce or change authentication, authorisation, RBAC, untrusted-input parsing on the wire, or cryptographic boundaries. The bug-shape concern around silent acceptance of invalid search params is an integrity-of-results question for the search engine, not an access-control or input-parsing vulnerability. Customer-facing permission decisions are unaffected.

Operational and observability risk concentrates on whatever explains the silent acceptance of invalid ef/nprobe/reorder_k values. If server-side validation was removed elsewhere, the production-side gap is the real concern but it lives outside this PR. A reviewer who notices the assertion direction in this PR should open a follow-up issue against the search-param validator (or whichever production layer used to emit the ef(7) should be larger than k(10) error) rather than treat this as a test-only PR with no production implication.

Test coverage gaps. The newly active tests assert happy-path behaviour for the new WithSearchParam API across a representative set of parameters (ef, nprobe, radius, range_filter, metric_type, drop_ratio_search, reorder_k). They do not cover the failure modes the test names imply they should cover. If the server should be validating these params and silently accepts them today, no test in this PR catches a regression where validation is removed or weakened in a future change. Worth listing as a follow-up rather than blocking on, since the PR's stated goal is to land the test-reactivation under the new API rather than to verify validation semantics.

Golden Answer Review

Golden Answer QC Feedback

****Grading Item 1 (3/3)****

The response is internally consistent, references specific behaviors, and avoids contradictions.

****Grading Item 2 (3/3)****

It is clearly grounded in repo exploration, detailing concrete files, tests, helper functions, and APIs from the codebase.

****Grading Item 3 (2/2)****

It directly answers the prompt: describes what the change is doing, which parts of the test suite and API are affected, and what issues to flag.

****Grading Item 4 (3/3)****

The reasoning chain is explicit, explaining how observations about the diff lead to conclusions about behavior, impacts, and reviewer concerns.

****Grading Item 5 (2/2)****

Technical explanations are precise, describing mechanisms (e.g., CheckErr semantics, WithSearchParam wiring) concretely.

****Grading Item 6 (2/2)****

All technical statements align with the repo context and behaviors, with no apparent inaccuracies.

****Grading Item 7 (3/3)****

All critical rubric requirements are met: enumerated files, production surfaces, security conclusion, alternative-fixes section, and detailed diff observations.

****Grading Item 8 (2/2)****

No penalty-triggering behaviors appear; the analysis avoids fabricated impacts or incorrect recommendations.

****Grading Item 9 (2/2)****

The coverage is comprehensive for the task, addressing major changes, ripples, and reviewer flags.

****Grading Item 10 (2/2)****

Writing is organized and understandable to non-experts, with clear headings and explanations.

****Grading Item 11 (2/2)****

Includes concrete details such as file paths, function names, helper behavior, and parameter values.

****Grading Item 12 (3/3)****

Follows PR Triage expectations, focusing on code changes, blast radius, severity, and security risk in an appropriate style.

****Grading Item 13 (2/2)****

Provides the four required PR Triage sections explicitly: code change summary, blast radius, severity/complexity, and security risk.

****Grading Item 14 (1/1)****

Contains only ASCII characters.

FINAL_SCORE: 32

Validate Golden Answer

Golden Answer Validation

```
{
  "results": [
    {
      "id": "reasoning-64318",
      "tag": "reasoning",
      "weight": "penalty",
      "description": "Recommends redesigning the CheckErr helper or the WithSearchParam API as the
primary triage outcome.",
      "rationale": "A redesign-first answer skips the actual review job. The change is a concrete diff to
approve or request-changes on, not a test-helper or builder-API exercise. Recommending a redesign
leaves the inverted-assertion question unanswered.",
      "dependent_on": [],
      "pass": true,
      "evidence": "The response analyzes the existing CheckErr helper but does not advise modifying the
helper or WithSearchParam API; instead it recommends specific test-level fixes such as adjusting
assertions and removing duplicate lines.",
      "reasoning": "The criterion is satisfied because the golden answer focuses on reviewing the diff and
suggesting test corrections rather than proposing a redesign of core helpers or APIs."
    },
    {
      "id": "reasoning-64312",
      "tag": "reasoning",
      "weight": "bonus",
      "description": "Notes that the 'but no error' suffix in the ef=7 CheckErr message wording documents an
observed behaviour change.",
      "rationale": "The message text only renders when CheckErr is invoked with expErrNil=false and an error
is present. Because expErrNil is true here the message is dead string. The author's wording ('ef(7) should
be larger than k(10), but no error') is therefore documentation of a behaviour change the author observed,
```


not an assertion of it. The smoking-gun observation distinguishes a careful read of the helper semantics from a surface read of the CheckErr call.",

```
"dependent_on": [  
  "reasoning-64303"  
],
```

```
"pass": true,
```

```
"evidence": "\u201cAfter both loops the test runs one more search with ef=7 ... and asserts ... The '\u201dbut no error\u201c suffix in that message wording is the smoking gun: the author knows ef=7 used to error and is documenting that it no longer does.\u201d",
```

```
"reasoning": "The answer explicitly highlights the 'but no error' wording as evidence that the author is documenting an observed change in behaviour rather than asserting it, satisfying the criterion."
```

```
},
```

```
{
```

```
  "id": "reasoning-64308",
```

```
  "tag": "reasoning",
```

```
  "weight": "bonus",
```

```
  "description": "Notes the new three-argument NewSCANNIndex(metricType, 16, withRawData) wiring in TestSearchScannAllMetricsWithRawData.",
```

```
  "rationale": "The pre-PR call was index.NewSCANNIndex(entity.COSINE, 16) inside the skipped /* ... */ block. The current production signature at client/index/scann.go:44 takes three arguments. Naming the wire-up shows the reviewer understood why the test could only run after the skip lifted: the body's prior 2-arg call would not compile against current production.",
```

```
  "dependent_on": [],
```

```
  "pass": true,
```

```
  "evidence": "\u201cTestSearchScannAllMetricsWithRawData \u2026 the index constructor is updated from index.NewSCANNIndex(entity.COSINE, 16) to the new three-argument index.NewSCANNIndex(metricType, 16, withRawData)\u201d and \u201cThe new SCANN tests reach \u2026 NewSCANNIndex(metricType MetricType, nlist int, withRawData bool)\u2026 Pre-PR the test code called the two-argument form\u2026 The reactivation lines up the test with the production three-argument signature.\u201d",
```

```
  "reasoning": "The golden answer explicitly notes that the SCANN test now uses the three-argument NewSCANNIndex(metricType, 16, withRawData) signature and contrasts it with the previous two-argument version, demonstrating clear understanding of the wiring change."
```

```
},
```

```
{
```

```
  "id": "reasoning-64304",
```

```

    "tag": "reasoning",
    "weight": "critical",
    "description": "States that the PR makes no changes to production code.",
    "rationale": "Anchoring the triage in the test-only scope bounds the severity ceiling. A reviewer who
imagines production-side risk inflates the severity and asks for review of code that does not move in this
patch.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Production code does not move.\" and later \"That is the entire diff. No production code
is touched.\"
    \"reasoning\": \"The golden answer explicitly states multiple times that the PR only affects tests and that
no production code files change, satisfying the criterion.\"
  },
  {
    \"id\": \"reasoning-64301\",
    \"tag\": \"reasoning\",
    \"weight\": \"critical\",
    \"description\": \"Identifies that TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex are
deleted as skip-only stubs and their scaffolding is merged into TestSearchInvalidSearchParams.\",
    \"rationale\": \"The consolidation is the load-bearing refactor in search_test.go. Without naming the
deletions and the merge a reader cannot judge that the new TestSearchInvalidSearchParams body was
previously distributed across three function names. The observation also bounds the test-count claim
correctly: three functions become one.\",
    \"dependent_on\": [],
    \"pass\": true,
    \"evidence\": \"\u201cThe two skip-only stub functions TestSearchEfHnsw and
TestSearchSearchParamsMismatchIndex are deleted entirely, and their setup scaffolding ... is merged
into TestSearchInvalidSearchParams which previously had only a t.Skip line.\u201d\",
    \"reasoning\": \"The response explicitly states that both skip-only stubs are removed and their scaffolding
combined into TestSearchInvalidSearchParams, satisfying the requirement to describe the consolidation.\"
  },
  {
    \"id\": \"reasoning-64305\",
    \"tag\": \"reasoning\",
    \"weight\": \"critical\",

```

"description": "Identifies the migration to the new WithSearchParam(key, value) builder API on searchOption defined in client/milvusclient/read_options.go.",

"rationale": "The reactivated and new tests reach the production search-param wire format through .WithSearchParam(...) calls (radius, range_filter, metric_type, ef, nprobe, drop_ratio_search, reorder_k). Naming the builder is what connects the un-skip pattern to the actual API unlock the PR is exercising for the first time in this acceptance path.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cThe change that ripples furthest is the wiring of the new WithSearchParam API. The builder method is defined twice in \u2018client/milvusclient/read_options.go\u2019: on the underlying annRequest at line 248 and on the public searchOption at line 354, both with signature WithSearchParam(key, value string).\u201d",

"reasoning": "The golden answer explicitly identifies the migration to the WithSearchParam builder API, referencing its definition in client/milvusclient/read_options.go and explaining how it is used, thus satisfying the criterion."

},

{

"id": "reasoning-64307",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies the semantic flip in TestQueryOutputFields where MergeColumnsToDynamic switches from GenDynamicColumnData(0, 10) to an empty column slice.",

"rationale": "MergeColumnsToDynamic at data_helper.go iterates the columns slice per row to assemble each row's dynamic JSON object. The empty slice yields empty per-row maps. The test moves from asserting a populated dynamic-field response (ten rows of generated data) to asserting empty dynamic data when the requested output field name does not exist. This is a spec change rather than a refactor and warrants a one-line note from the author.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cthe dynamicColumn construction is changed from hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), common.DefaultDynamicFieldName) to hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)\u2026With an empty slice every row's map is empty. The test thereby moves from asserting that the query returns ten rows of populated dynamic data to asserting that \u2026 returns \u2026 empty dynamic data per row.\u201d",

"reasoning": "The golden answer explicitly explains that MergeColumnsToDynamic now receives an empty column slice, producing empty dynamic data and altering the test's semantics, satisfying the criterion."

},

{

"id": "reasoning-64309",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes that t.Parallel is added to TestSearchGroupByFloatGrowing and TestSearchGroupByPagination but not to TestSearchGroupByBinaryDefault in the same file.",

"rationale": "The parallelism decision is inconsistent across three otherwise-similar growing tests. Whether to parallelise is a fair call, but the asymmetry reads as oversight rather than a deliberate split. A reviewer who notes the asymmetry is checking that t.Parallel additions are systematic rather than incidental.",

"dependent_on": [],

"pass": true,

"evidence": "Two tests gain a t.Parallel() call: TestSearchGroupByFloatGrowing and TestSearchGroupByPagination. The third growing test in the file, TestSearchGroupByBinaryDefault, does not get t.Parallel(), so the parallel-coverage decision is inconsistent.",

"reasoning": "The golden answer explicitly points out the asymmetric addition of t.Parallel calls across the three growing tests and comments on the inconsistency, fulfilling the criterion."

},

{

"id": "reasoning-64313",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes the log.Debug to log.Info change in TestQueryOutputFields that promotes per-row dynamic-field data to standard CI output.",

"rationale": "Promoting debug logs to info level is a noise change. CI runs will surface the row dump in standard log output going forward. Whether that is wanted is a one-line question from the author. A reviewer who flags it is reading the diff for noise-level changes rather than only assertion changes.",

"dependent_on": [],

"pass": true,

"evidence": "\"Inside the enableDynamic branch, log.Debug is changed to log.Info so the debug-level row dump is now always printed\" and later \"log.Debug to log.Info in TestQueryOutputFields is a noise

change that will surface row dumps in every CI run; whether that is wanted is worth one line from the author.\""

"reasoning": "The golden answer explicitly notes the log level change in TestQueryOutputFields, including its effect on CI output, satisfying the criterion."

},

{

"id": "reasoning-64302",

"tag": "reasoning",

"weight": "critical",

"description": "Identifies that the new TestSearchInvalidSearchParams asserts success for invalid ef and nprobe values via CheckErr with expErrNil=true.",

"rationale": "The CheckErr helper at response_checker.go:19-41 treats the second argument as expErrNil and the third as an unused message when expErrNil is true. The new body issues searches with invalidEfs {-1, 0, 32769} and invalidNprobes {-1, 0, 65537} and asserts each call returns no error. The test name promises invalid-param coverage; the assertions document silent acceptance. Without surfacing this contradiction a reviewer ships an assertion direction that points the wrong way.",

"dependent_on": [],

"pass": true,

"evidence": "\u201cMost consequential: the new bodies of TestSearchInvalidSearchParams and TestSearchInvalidScannReorderK assert success, not failure, for cases the test names describe as 'invalid'. Each invalidEf in {-1, 0, 32769} is exercised with common.CheckErr(t, errHnsw, true, 'No error for invalid search params'). The expErrNil=true argument means require.NoError\u2026 The same pattern repeats for invalidNprobes in {-1, 0, 65537}.\u201d",

"reasoning": "The golden answer explicitly explains that TestSearchInvalidSearchParams now uses CheckErr with expErrNil=true for invalid ef and nprobe values, highlighting that the tests assert success despite the name promising invalid-param coverage."

},

{

"id": "reasoning-64315",

"tag": "reasoning",

"weight": "penalty",

"description": "Asserts that the PR introduces a classic security regression (authentication, authorisation, RBAC, untrusted-input parsing, or network surface).",

"rationale": "The diff is acceptance-test reactivation and consolidation. Asserting a classic security issue inflates the severity and misroutes the fix toward production layers that do not move in this patch.",

"dependent_on": [],

```
"pass": true,
  "evidence": "\u201cNo classic security regression. The diff does not introduce or change authentication, authorisation, RBAC, untrusted-input parsing on the wire, or cryptographic boundaries.\u201d",
  "reasoning": "The golden answer explicitly states that there is no classic security regression, so it does not exhibit the harmful behavior of asserting one."
},
{
  "id": "reasoning-64306",
  "tag": "reasoning",
  "weight": "critical",
  "description": "Identifies that TestSearchGroupByRangeSearch and TestRangeSearchSparseVector are re-skipped under issue 38846 with their bodies migrated to the new WithSearchParam API.",
  "rationale": "The re-skip pattern is easy to miss: the t.Skip is preserved but the body is now fully implemented and waits for #38846 to be resolved. A reviewer who reads only the t.Skip line treats the tests as inactive without noticing that the API migration work landed under the skip guard.",
  "dependent_on": [],
  "pass": true,
  "evidence": "\"TestSearchGroupByRangeSearch is re-skipped under a different issue (now Skipf(\">https://github.com/milvus-io/milvus/issues/38846\") but its body is rewritten... updated ... to the new chained .WithSearchParam...\" and \"TestRangeSearchSparseVector loses its old skip and is re-skipped under issue #38846 with a fully-implemented body: a default search using WithSearchParam...\"",
  "reasoning": "The golden answer explicitly notes both tests being re-skipped under issue 38846 and describes their bodies being migrated to the WithSearchParam API, satisfying the criterion."
},
{
  "id": "reasoning-64314",
  "tag": "reasoning",
  "weight": "penalty",
  "description": "Asserts as fact that the PR modifies production code (the WithSearchParam builder, the NewSCANNIndex constructor, or the server-side search-param validator).",
  "rationale": "The PR is test-only across three test files. Claiming production-side changes misreads the diff entirely and would have a reviewer ask for review of code that does not move in this patch.",
  "dependent_on": [],
  "pass": true,
```

```

    "evidence": "\"That is the entire diff. No production code is touched.\",",
    "reasoning": "The golden answer explicitly states that no production code is modified and never claims
otherwise, thus avoiding the prohibited assertion."
},
{
    "id": "reasoning-64319",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Recommends splitting the un-skip work from the consolidation refactor as a hard
blocker.",
    "rationale": "Splitting at review time adds two PRs of overhead without producing a behaviour change.
The consolidation is what makes the un-skips coherent. The right outcome is to ask the author to call out
the consolidation in the PR description, not to demand a split.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Alternative fixes that are tempting but wrong. ... Splitting the un-skip work from the
consolidation refactor as a hard block is wrong because the consolidation is what makes the un-skips
coherent...\",",
    "reasoning": "The golden answer explicitly argues against demanding a split of the un-skip work from
the consolidation refactor, so it does not recommend the harmful behavior."
},
{
    "id": "reasoning-64303",
    "tag": "reasoning",
    "weight": "critical",
    "description": "Identifies that the ef=7 case in TestSearchInvalidSearchParams asserts success even
though DefaultLimit=10 and the historical validation rule was ef should be larger than k.",
    "rationale": "The pre-PR equivalent assertion was CheckErr(t, err, false, 'ef(7) should be larger than
k(10)') which expected the error. The post-PR assertion flips to expErrNil=true with message 'ef(7) should
be larger than k(10), but no error', so the message documents the behaviour change in plain prose. A
reviewer who reads only the literal CheckErr call without unpacking the helper semantics misses the
inverted assertion entirely.",
    "dependent_on": [
        "reasoning-64302"
    ],
    "pass": true,

```

"evidence": "\u201cAfter both loops the test runs one more search with ef=7 (where DefaultLimit=10 so ef<topK by the historical validation rule) and asserts common.CheckErr(t, err, true, \"ef(7) should be larger than k(10), but no error\"). The \"but no error\" suffix in that message wording is the smoking gun: the author knows ef=7 used to error and is documenting that it no longer does.\u201d",

"reasoning": "The golden answer explicitly highlights that the ef=7 check now expects success despite historically requiring ef>k, noting both the DefaultLimit=10 context and how the new assertion documents the deviation."

},

{

"id": "reasoning-64310",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes the dead commented-out MergeColumnsToDynamic call left in TestQueryOutputFields immediately above the new live call.",

"rationale": "Leaving a one-line comment that mirrors the previous behaviour adds no value and rots quickly. A reviewer who flags the dead-code line is doing the lowest-cost housekeeping ask the PR would benefit from. The observation also paired well with the wider concern that the semantic flip in this test needs a PR-description note.",

"dependent_on": [],

"pass": true,

"evidence": "The leftover commented-out hp.MergeColumnsToDynamic(10, hp.GenDynamicColumnData(0, 10), ...) line should be removed before merge regardless because dead code in test files rots quickly.",

"reasoning": "The answer explicitly notes the commented-out MergeColumnsToDynamic call and recommends removing it, satisfying the criterion."

},

{

"id": "reasoning-64311",

"tag": "reasoning",

"weight": "bonus",

"description": "Notes the duplicate common.CheckErr(t, errSearch, true) line on consecutive lines inside TestRangeSearchSparseVector.",

"rationale": "The duplicate is a clear copy-paste artefact. It runs twice harmlessly today but degrades readability and would surface if someone changed one of the two without realising they had a sibling. A reviewer who catches the duplicate is reading the body of a re-skipped test rather than skipping past it on the t.Skip line.",


```

    "dependent_on": [],
    "pass": true,
    "evidence": "The body contains a duplicate common.CheckErr(t, errSearch, true) line on consecutive
lines, which is a copy-paste artefact.",
    "reasoning": "The golden answer explicitly highlights the duplicate CheckErr line in
TestRangeSearchSparseVector, satisfying the requirement to note this issue."
  },
  {
    "id": "reasoning-64317",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Recommends preserving the deleted TestSearchEfHnsw and
TestSearchSearchParamsMismatchIndex stubs for git-blame continuity.",
    "rationale": "Both deleted functions were skip-only stubs with no test logic. Preserving them after their
content has been merged into TestSearchInvalidSearchParams adds maintenance cost without producing
test coverage. The consolidation is housekeeping a reviewer should encourage.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Demanding the deleted TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex
stubs be preserved for git-blame continuity is wrong; deleting unused skip-only functions is housekeeping
and the merge into TestSearchInvalidSearchParams is exactly the kind of consolidation a reviewer should
encourage.\"\"",
    "reasoning": "The golden answer explicitly discourages preserving the deleted stubs, aligning with the
rubric and avoiding the penalized behavior."
  },
  {
    "id": "reasoning-64300",
    "tag": "reasoning",
    "weight": "critical",
    "description": "Identifies that three previously-skipped tests in
tests/go_client/testcases/groupby_search_test.go lose their issue-38343 t.Skip guards and become
active (TestSearchGroupByFloatDefault, TestSearchGroupByFloatDefaultCosine,
TestSearchGroupByBinaryGrowing).",
    "rationale": "The un-skipping is the headline activation in the GroupBy file. Anchoring the triage on the
three named tests is how the reader connects the issue-38343 reference to the concrete CI coverage that
just turned on.",

```

```

    "dependent_on": [],
    "pass": true,
    "evidence": "\"Three previously-skipped tests lose their
t.Skip(\\\"https://github.com/milvus-io/milvus/issues/38343\\\") guards and become active:
TestSearchGroupByFloatDefault, TestSearchGroupByFloatDefaultCosine, and
TestSearchGroupByBinaryGrowing.\\\"",
    "reasoning": "The golden answer explicitly identifies all three specified tests in groupby_search_test.go
as having their issue-38343 skips removed and reactivated, satisfying the criterion directly."
},
{
    "id": "reasoning-64316",
    "tag": "reasoning",
    "weight": "penalty",
    "description": "Recommends reverting the un-skip changes as the appropriate fix for the
inverted-assertion concern in TestSearchInvalidSearchParams.",
    "rationale": "The un-skips are correct work. Only the assertion direction inside the new body is the
question, and the right fix is to flip the second CheckErr argument from true to false if the intent is to
assert rejection, or to rename the test if the intent is to document silent acceptance. Reverting throws
away the consolidation and the migration to the new builder API.",
    "dependent_on": [],
    "pass": true,
    "evidence": "\"Reverting the un-skip changes because the assertions look inverted is wrong: the
un-skips are correct work...\\\"",
    "reasoning": "The golden answer explicitly states that reverting the un-skip changes would be incorrect,
thereby avoiding the penalized behavior."
},
{
    "id": "completeness-64321",
    "tag": "completeness",
    "weight": "critical",
    "description": "Names the production surface the new tests reach without the PR touching it
(WithSearchParam builder, NewSCANNIndex three-argument constructor, server-side search-param
validator).",
    "rationale": "The diff is test-only and the reactivated tests reach production code the PR leaves intact. A
triage that does not name the production surface cannot ground the blast-radius claim that the new tests

```

exercise unmodified code. The reader needs the surface named to verify the no-production-change claim.",

```
"dependent_on": [],
```

```
"pass": true,
```

```
"evidence": "\"The change that ripples furthest is the wiring of the new WithSearchParam API. The builder method is defined twice in \\\"client/milvusclient/read_options.go\\\" ...\\\"; \\\"The new SCANN tests reach \\\"client/index/scann.go\\\" ... NewSCANNIndex(metricType MetricType, nlist int, withRawData bool)\\\"; \\\"Every newly-active or migrated test in this PR is now reaching the server-side validator ...\\\"\",
```

```
"reasoning": "The answer explicitly identifies each untouched production surface impacted by the tests\u2014WithSearchParam builder, three-argument NewSCANNIndex, and the server-side search-parameter validator\u2014meeting the criterion."
```

```
},
```

```
{
```

```
"id": "completeness-64323",
```

```
"tag": "completeness",
```

```
"weight": "critical",
```

```
"description": "Includes an alternative-fixes paragraph preempting wrong-fix temptations such as reverting the un-skip changes, preserving the deleted stubs, or splitting the un-skip from the consolidation.",
```

```
"rationale": "Preempting wrong fixes is what makes the Severity section actionable. Showing the reviewer thought through alternatives before settling on the recommendation distinguishes a triage from a bullet list of complaints.",
```

```
"dependent_on": [],
```

```
"pass": true,
```

```
"evidence": "\u201cAlternative fixes that are tempting but wrong. Reverting the un-skip changes because the assertions look inverted is wrong\u201d Demanding the deleted TestSearchEfHnsw and TestSearchSearchParamsMismatchIndex stubs be preserved\u201d is wrong\u201d Splitting the un-skip work from the consolidation refactor\u201d is wrong\u201d\u201d",
```

```
"reasoning": "The answer contains a dedicated paragraph explicitly preempting wrong fixes, discussing why reverting un-skips, preserving deleted stubs, or splitting the work would be incorrect, satisfying the criterion."
```

```
},
```

```
{
```

```
"id": "completeness-64320",
```

```
"tag": "completeness",
```

```
"weight": "critical",
```

```

    "description": "Enumerates the three test files changed by the diff.",
    "rationale": "The mechanical surface of the PR spans three files under tests/go_client/testcases
(groupby_search_test.go, query_test.go, search_test.go). A triage that does not enumerate the files is
incomplete because downstream blast-radius and severity claims rest on knowing which surfaces
moved.",
    "dependent_on": [],
    "pass": true,
    "evidence": "The answer explicitly states: \"\"tests/go_client/testcases/groupby_search_test.go\" gets
four kinds of edit...\"; \"\"tests/go_client/testcases/query_test.go\" gets a single test reactivated...\"; and
\"\"tests/go_client/testcases/search_test.go\" gets the bulk of the change.\"",
    "reasoning": "It clearly enumerates all three modified test files, satisfying the requirement to identify the
mechanical surface of the PR."
  },
  {
    "id": "completeness-64324",
    "tag": "completeness",
    "weight": "penalty",
    "description": "Centers the response on a redesign proposal (proposing a new CheckErr helper,
restructuring the WithSearchParams builder, or rewriting the test-organisation pattern) rather than triaging
the diff as written.",
    "rationale": "A redesign-first answer skips the actual review job. The change is a concrete diff to
approve or request-changes on, and an answer that pivots to design exercises does not produce an
approval decision the author can act on.",
    "dependent_on": [],
    "pass": true,
    "evidence": "The response concentrates on triaging the diff, summarizing the code changes, blast
radius, and specific issues to flag (e.g., inverted assertions, duplicate CheckErr).",
    "reasoning": "The answer does not pivot to proposing new helper designs or architectures; it remains
focused on evaluating the existing change and what needs attention before merge."
  },
  {
    "id": "completeness-64322",
    "tag": "completeness",
    "weight": "critical",
    "description": "Concludes the PR introduces no classic security regression.",

```

"rationale": "Test-only PRs invite the lazy reading that drops the security analysis entirely. Explicitly concluding no classic regression is what makes the security framing complete and distinguishes a thorough triage from one that omits the security frame.",

"dependent_on": [],

"pass": true,

"evidence": "Security Risk\\n\\nNo classic security regression.",

"reasoning": "The golden answer explicitly states that there is no classic security regression, directly satisfying the criterion requiring such a conclusion."

},

{

"id": "style-64325",

"tag": "style",

"weight": "bonus",

"description": "Organises the response into the four PRT&IA sections (Code Change Summary, Blast Radius, Severity and Complexity, Security Risk) or uses equivalent named sections that cover the same content.",

"rationale": "The four-section structure is the canonical PRT&IA output shape and helps the reader absorb a multi-concern review without losing track of which concern lives where.",

"dependent_on": [],

"pass": true,

"evidence": "The response explicitly uses sections titled \\u201cCode Change Summary,\\u201d \\u201cBlast Radius,\\u201d \\u201cSeverity and Complexity,\\u201d and \\u201cSecurity Risk.\\u201d",

"reasoning": "The golden answer mirrors the requested four-section structure, ensuring the content is organised into the canonical PRT&IA sections."

},

{

"id": "style-64326",

"tag": "style",

"weight": "bonus",

"description": "Uses plain-text identifiers for function and type names and double-quoted file paths.",

"rationale": "The project's prose convention pairs plain-text identifiers with double-quoted paths.

Consistent formatting helps the reader visually parse references at a glance and avoids the generated-artifact feel of backtick-wrapped tokens.",

"dependent_on": [],

"pass": true,

```

    "evidence": "File paths such as \"tests/go_client/testcases/groupby_search_test.go\" and
\"client/milvusclient/read_options.go\" are enclosed in double quotes, and identifiers like
TestSearchGroupByFloatDefault and TestSearchInvalidSearchParams appear in plain text.",
    "reasoning": "All references to functions and file paths follow the stated convention: identifiers are plain
text and paths are double-quoted."
  }
],
"passed": 27,
"failed": 0,
"total": 27,
"score_percent": 100.0,
"summary": "Passed 27/27 criteria (100.0%)"
}

```

Test Model

```

=====
RUN 1/3
=====
### What actually changed beyond "unlock previously-skipped Go client search tests"?

```

The patch does more than just drop a few `t.Skip`` calls:

- Group-by search coverage is back on**
 - ``TestSearchGroupByFloatDefault``, ``TestSearchGroupByFloatDefaultCosine``, and ``TestSearchGroupByBinaryGrowing`` are now active again after removing the skip that referenced issue #38343 (see ``tests/go_client/testcases/groupby_search_test.go``, lines 92-199 and 297-327).
 - ``TestSearchGroupByFloatGrowing`` and ``TestSearchGroupByPagination`` were also marked ``t.Parallel()`` so they run concurrently (lines 330-405).
 - The range-search test still skips, but the reason was updated and the body now shows how ``WithSearchParam("radius", ...) / ("range_filter", ...)`` will be used once issue #38846 is fixed (lines 443-459).
- The previously skipped query test is live**
 - ``TestQueryOutputFields`` no longer calls ``t.Skip`` and the expected dynamic-field payload was altered (see ``tests/go_client/testcases/query_test.go``, lines 165-236). The change swaps ``hp.MergeColumnsToDynamic(.., hp.GenDynamicColumnData(..)`` for an empty dynamic column when asserting output for a fake field.
- Large portions of ``search_test.go`` now exercise real search parameters**

* Additional tests now pass explicit search parameters to the client, e.g. ``WithSearchParam("metric_type", "L2")`` in ``TestSearchNotMatchMetricType`` (lines 455-469), HNSW ``ef`/`nprobe`` checks plus SCANN ``reorder_k`` (lines 509-580), and sparse-vector range-search params (lines 1061-1095).

* ``TestSearchScannAllMetricsWithRawData`` is fully enabled again and loops through metrics and the ``with_raw_data`` flag (lines 555-581).

* All these tests rely on ``client.NewSearchOption(...).WithSearchParam(key, value)``, which feeds the ``searchParam`` map in ``client/milvusclient/read_options.go`` (lines 248-356). That map is merged into ``SearchParams`` in the generated protobuf request (lines 103-139), so the API surface being covered is the existing ``WithSearchParam`` helper on ``SearchOption`/`annRequest``.

4. **Range-search tests for sparse vectors now show the intended API usage**

Even though ``TestRangeSearchSparseVector`` still skips pending server support, it now demonstrates how ``drop_ratio_search``, ``radius``, and ``range_filter`` parameters will be supplied once enabled (lines 1079-1105).

Ripple effects on the Go client surface

Although no production code changed, the tests now actively exercise a wider slice of the Go SDK:

- * Search options now ensure that arbitrary ANN/search params (metric type, ef, nprobe, radius, filters, SCANN reorder_k, sparse drop ratios, etc.) get serialized through ``WithSearchParam`` (see ``client/milvusclient/read_options.go``, lines 248-356).
- * Group-by search coverage stresses combinations of ``WithGroupByField``, ``WithANNSField``, ``WithConsistencyLevel``, and the concurrency of those calls.
- * Query output-field handling (dynamic field projection vs. fake columns) is exercised more thoroughly in ``TestQueryOutputFields``.

Issues to raise before approving

1. **Dynamic-field expectation now contradicts the inserted data**

In ``TestQueryOutputFields``, the expected column for the fake field ``aaa`` is built from ``hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)`` (lines 193-210).

*The inserted rows still include real dynamic columns (because ``hp.CollPrepare.InsertData`` automatically appends ``hp.GenDynamicColumnData``).

As a result, the expected JSON (``{}`` for every row) will never match the actual dynamic payload (``dynamicNumber``, ``dynamicString``, etc.). This test will fail as soon as CI runs it. The previous code (now commented out) that merged the generated dynamic columns was correct; it should be restored.

2. **“Invalid search params” tests now expect success instead of failure**

In ``TestSearchInvalidSearchParams`` (lines 521-544) the new loops call ``common.CheckErr(t, err, true, "No error for invalid search params")`` even though the comments say the calls *should* error. Passing ``true`` to ``CheckErr`` asserts that no error occurred, so any server-side validation will make the test fail for the wrong reason. The same mistake appears again when searching with ``ef=7``, which is explicitly documented as invalid (lines 539-544). The expectation flags need to be flipped to ``false``.

3. ****SCANN invalid-parameter test has the same logic bug****

`TestSearchInvalidScannReorderK` now asserts success for the invalid `reorder\_k < topK` case (lines 547-558). If the server correctly rejects that request, this test will fail. The expectation should be `common.CheckErr(t, errScann, false, ...)`.

4. ****Verify that the un-skipped group-by binary test is actually expected to pass****

`TestSearchGroupByBinaryGrowing` still asserts the server should ***reject*** binary vector group-by (lines 297-327). It was previously skipped for issue #38343; if that server issue is still open, the test will immediately fail. Confirm with the feature owner before turning it on in CI.

Addressing the three logic issues above (dynamic field expectations and the two invalid-search-param tests) is necessary before the patch can be considered safe to merge; otherwise the newly enabled tests will either fail outright or assert the opposite of the intended behavior.

```
=====
RUN 2/3
=====
### What the patch really does
```

1. ****Enables the Go client to exercise per-search overrides****

All of the newly-unskipped tests now call `client.NewSearchOption(...).WithSearchParam(key, value)` so the SDK pushes arbitrary key/value pairs down to the server. Internally (`client/milvusclient/read\_options.go`, lines 115-176, 230-276) these keys are merged into the `searchParam` map that becomes the `SearchRequest.SearchParams` payload. Those entries override the defaults that were previously hard-wired by the index definition (e.g., HNSW `ef`, IVF `nprobe`, SCANN's `reorder\_k`, sparse range parameters such as `drop\_ratio\_search`, `radius`, `range\_filter`, and even `metric\_type`). Until now that code path existed but none of the regression tests actually drove it.

2. ****Re-enables the Go test suites that used to be blocked on "missing search param support."****

* `tests/go\_client/testcases/groupby\_search\_test.go` – the float/cosine group-by suites and the binary growing suite now run (lines 92, 150, 298), `TestSearchGroupByFloatGrowing` / `...Pagination` are also marked `t.Parallel` (lines 329 and 383), and the "range search + group by" negative test now explicitly sends `radius` / `range\_filter` even though it still skips via issue #38846 (lines 442-455).

* `tests/go\_client/testcases/search\_test.go` – the previously skipped blocks (`TestSearchNotMatchMetricType`, `TestSearchInvalidSearchParams`, `TestSearchInvalidScannReorderK`, `TestSearchScannAllMetricsWithRawData`, `TestRangeSearchSparseVector`) are now active and drive the various `WithSearchParam` combinations (lines 468-1112). This not only covers metric mismatch but also validates invalid `ef`, `nprobe`, SCANN's `reorder\_k`, sparse range searches, and the SCANN `with\_raw\_data` flag across all supported metrics.

* `tests/go\_client/testcases/query\_test.go` – `TestQueryOutputFields` is no longer skipped (lines 167-236) so the client's `\$meta` / dynamic-field handling is now exercised.

3. ****Small cleanup around logging and skipping****

* Query test logging switched from `log.Debug` to `log.Info` when printing the field payloads (line 201).

* The remaining range-search tests are now tied to the tracking issue (#38846) rather than the old “waiting for support” string.

Ripple across the Go client surface

* **Search parameter API** – WithSearchParam is now used in real tests for:

- Metric overrides (``metric_type``), HNSW tuning (``ef``), IVF (``nprobe``), SCANN's runtime controls (``reorder_k``, ``with_raw_data``), sparse range parameters (``drop_ratio_search``, ``radius``, ``range_filter``), and even restricting group-by + range search.
- These tests indirectly verify the SDK's ability to merge those overrides into the proto request via ``annRequest.searchRequest()`` (see ``client/milvusclient/read_options.go``, lines 115-176).

* **Group-by search** – The float, cosine, binary, growing, and pagination scenarios now execute end-to-end (tests/go_client/testcases/groupby_search_test.go, lines 92-404). They all rely on the search-param surface to keep index-specific knobs in sync (e.g., HNSW on growing segments). Removing the skip means any regression in per-search overrides will now fail these suites.

* **Query dynamic output handling** – Because ``TestQueryOutputFields`` now runs, the SDK's logic that maps “unknown” output field names to ``$meta`` slices (``client/milvusclient/read.go``, lines 109-175) finally has coverage.

Issues to flag before approving

1. **The “invalid search param” tests currently assert success instead of failure.**

* In ``TestSearchInvalidSearchParams`` and ``TestSearchInvalidScannReorderK``, every negative case calls ``common.CheckErr(t, err, true, "No error for invalid search params")`` (search_test.go lines 525-538 and 559-567). ``expErrNil=true`` tells ``CheckErr`` to require **no error**, so the test would happily pass even if the server silently accepted illegal ``ef``, ``nprobe``, or ``reorder_k`` values. The intent (per the comments) is the opposite. Please flip those expectations to ``false``, or these tests will never catch the regressions they're meant to guard.

2. **``TestQueryOutputFields`` now asserts empty dynamic payloads even though the fixture inserts dynamic data.**

* The new expectation builds ``dynamicColumn := hp.MergeColumnsToDynamic(10, []column.Column{}, common.DefaultDynamicFieldName)`` (query_test.go lines 199-208), i.e., an array of ``{}`` documents. But the helper ``CollPrepare.InsertData`` still injects real dynamic columns whenever ``enableDynamic == true`` (see ``tests/go_client/testcases/helper/data_helper.go``, lines 206-236). The actual server response for ``WithoutOutputFields("aaa")`` should therefore wrap the real ``$meta`` JSON, not an empty map. This change either masks a server-side bug or makes the test incorrect; worth clarifying before we rely on it.

3. **Potential flakiness/perf regressions from enabling very heavy tests in parallel.**

* ``TestSearchGroupByFloatDefault``, ``...Cosine``, ``...BinaryGrowing``, ``...FloatGrowing``, ``...Pagination``, plus the SCANN and sparse range suites, all iterate over multiple index types and NQ batches (100×200 inserts, repeated per metric). Running them all (``t.Parallel``) increases total load/CPU significantly

compared to the previously skipped state. Make sure CI capacity and timeouts are adjusted accordingly, or be prepared for flakes/timeouts when multiple of these run concurrently.

Addressing (1) is critical because it undermines the safety net we're trying to add; (2) needs clarification so we don't bake an incorrect expectation into the suite; and (3) is operational but should be acknowledged before merging.

Once those are resolved/acknowledged, the patch does achieve its headline goal: the SDK's customizable search-parameter plumbing now has real test coverage, and the group-by/search suites are no longer artificially blocked.

=====

RUN 3/3

=====

****What changed beyond the headline****

***Group-by search tests*` (tests/go\_client/testcases/groupby\_search\_test.go`)**

- The three float-based group-by suites (`TestSearchGroupByFloatDefault`, `...Cosine`, and `...BinaryGrowing`) no longer short-circuit with `t.Skip` so they now execute end-to-end, exercising all supported float and binary index combinations (lines 90-329).
- The "growing" and pagination variants now run with `t.Parallel()` (lines 329-345, 382-389), so multiple heavy data-setup tests execute concurrently.
- The range-search case for group-by is still skipped, but the body now demonstrates how to attach per-search parameters via `.WithSearchParam("radius", ...).WithSearchParam("range\_filter", ...)` (lines 443-454).

***Query output-fields test*` (tests/go\_client/testcases/query\_test.go`)**

- `TestQueryOutputFields` is now active (skip removed at line ~165) and its expected dynamic-field column is built from an empty dynamic source (`hp.MergeColumnsToDynamic(10, []column.Column{}, "\$meta")` at lines 193-206). That change reflects the behavior when requesting a field name that only exists in `\$meta`: the SDK should return an empty column rather than synthetic data.

***Search tests*` (tests/go\_client/testcases/search\_test.go`)**

- The suite imports `strconv` to format numeric search parameters (line 6).
- Previously skipped tests now run and explicitly drive the search-parameter API:
 - * `TestSearchNotMatchMetricType` sets `metric\_type` through `.WithSearchParam("metric\_type", "L2")` to verify mismatch handling (lines 455-470).
 - * `TestSearchInvalidSearchParams` now issues searches that override `ef` and `nprobe` via `.WithSearchParam`, and also tries the "ef < topK" case (lines 508-540).
 - * `TestSearchInvalidScannReorderK` sets SCANN-specific `nprobe`/`reorder\_k` pairs (lines 542-567).
 - * `TestSearchScannAllMetricsWithRawData` now actually iterates over metric types and the `with\_raw\_data` toggle, pulling `\*` output fields (lines 570-598).
 - * `TestRangeSearchSparseVector` no longer skips; it attaches sparse-vector-specific parameters (`drop\_ratio\_search`, `radius`, `range\_filter`) and asserts the returned scores fall inside the requested window (lines 1061-1097).

Collectively these tests exercise the `milvusclient.SearchOption.WithSearchParam`` and `annRequest.searchParam`` plumbing defined in `client/milvusclient/read_options.go`` (lines 83-138 & 248-356). They demonstrate that callers can now provide arbitrary key/value overrides (metric type, ef, nprobe, SCANN knobs, sparse range-search knobs, etc.) when constructing `SearchOption`s`.

****Surface area affected****

- ****Go client API****: no code changes in the client package itself, but the `WithSearchParam`` fluent API (both on `searchOption`` and `annRequest``) is now covered by concrete tests using metric, ef/nprobe, SCANN, and sparse-range parameters.
- ****Test suites****: group-by search, query output fields, and multiple search behaviors (metric mismatch, invalid params, SCANN, sparse range search) now run by default and, for the group-by cases, in parallel. The sparse range-search test also now verifies score bounds.

****Issues to flag before approval****

1. ****Invalid-parameter tests still expect success****

In `TestSearchInvalidSearchParams`` and `TestSearchInvalidScannReorderK``, the comments say “invalid ... -> error”, but the assertions call `common.CheckErr(t, err, true, "No error for invalid search params")``, i.e. they require `err == nil`` (lines 522-567). That means these tests currently *pass* when no validation happens* and will immediately fail if Milvus starts returning the expected errors. If the intent is to ensure invalid parameters are rejected, `expErrNil`` should be `false`` and the expected error message should be asserted.

2. ****Redundant/typo'd assertion in sparse range-search test****

`TestRangeSearchSparseVector`` calls `common.CheckErr(t, errSearch, true)`` twice in a row right after issuing the range search (lines 1103-1106). The second invocation is redundant (and still reads the same `errSearch``). While harmless, it's a sign the test might have been copy/pasted without cleaning up; it's worth trimming to avoid confusion.

3. ****Increased parallel load for heavy group-by tests****

Adding `t.Parallel()`` to `TestSearchGroupByFloatGrowing`` and `TestSearchGroupByPagination`` (lines 329-345 and 382-389) means each test now spins up large collections (hundreds of inserts and index builds) concurrently. On constrained CI environments this could exacerbate timeouts or resource contention. If CI instability shows up, we may need to gate these behind runtime controls.

If the expectation for item (1) is indeed “should error”, correcting the `CheckErr`` flags is important before declaring the tests “unblocked”, otherwise they provide a false sense of coverage.

Model Evaluation

```
{  
  "points": 32.7,  
  "maxPoints": 65.0,
```

```
"score_percent_weight": 50,
"summary_weight": "32.7/65.0 points (50%)",
"eval_runs": 3,
"eval_runs_succeeded": 3,
"per_run_points": [
  35.0,
  30.0,
  33.0
],
"results": [
  {
    "id": "reasoning-69416",
    "weight": "penalty",
    "pass": false,
    "points": 0.0
  },
  {
    "id": "reasoning-69397",
    "weight": "critical",
    "pass": false,
    "points": 0.0
  },
  {
    "id": "reasoning-69407",
    "weight": "critical",
    "pass": true,
    "points": 3.0
  },
  {
    "id": "reasoning-69418",
    "weight": "penalty",
    "pass": false,
    "points": 0.0
  },
  {
    "id": "reasoning-69410",
```

```
"weight": "bonus",
"pass": false,
"points": 0.0
},
{
  "id": "reasoning-69409",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69417",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69400",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69403",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69405",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69402",
```

```
"weight": "critical",
"pass": true,
"points": 3.0
},
{
  "id": "reasoning-69406",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69411",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69412",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69404",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69399",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69413",
```

```
"weight": "bonus",
"pass": false,
"points": 0.0
},
{
  "id": "reasoning-69401",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69398",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69414",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69396",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "reasoning-69415",
  "weight": "penalty",
  "pass": true,
  "points": -1.0
},
{
  "id": "reasoning-69408",
```

```
"weight": "critical",
"pass": true,
"points": 3.0
},
{
  "id": "reasoning-69420",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "reasoning-69419",
  "weight": "penalty",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-69424",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-69425",
  "weight": "critical",
  "pass": false,
  "points": 0.0
},
{
  "id": "completeness-69422",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "completeness-69423",
```



```
"weight": "critical",
"pass": false,
"points": 0.0
},
{
  "id": "completeness-69421",
  "weight": "critical",
  "pass": true,
  "points": 3.0
},
{
  "id": "style-69429",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "style-69428",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
},
{
  "id": "style-69430",
  "weight": "bonus",
  "pass": false,
  "points": 0.0
}
}
]
```

model eval score	50.3
difficulty	medium